

Mini Projeto – Sistema de Regras de Taxas Bancárias

Banco Digital NXT — Análise e Projeto (Fases 1 e 2)

Alunos: Arthur Carvalho, Maycon Vinicius e Nicolas Ferreira

FASE 1 — ANÁLISE DO CÓDIGO LEGADO

Respostas das 5 Perguntas

1. O que acontece se o programador digitar "Corrente" com letra maiúscula?

O método `String.equals()` é estritamente sensível a maiúsculas e minúsculas (*case-sensitive*). A comparação `tipoConta.equals("corrente")` retornará `false` para a entrada "Corrente", e nenhum dos blocos `if-else` será executado. O método simplesmente retornará 0.0 sem emitir nenhum erro, exceção ou aviso.

2. O que acontece se um novo tipo de transação ("BOLETO") for criado?

Nenhum dos blocos `if` internos contempla "BOLETO", portanto o retorno será sempre 0.0 — ou seja, o boleto seria processado sem cobrança de taxa, gerando prejuízo ao banco. Mais grave ainda: para corrigir isso, seria necessário entrar em cada um dos blocos de tipo de conta (corrente, poupança, universitária, black) e adicionar um novo `if` em cada um (quatro pontos de alteração já nas contas atuais, e mais um por cada nova conta que surgir no futuro).

3. Quantas linhas de código precisariam ser modificadas para adicionar uma Conta PJ?

Seria necessário modificar a classe `BancoLegado` em pelo menos dois lugares:

No método `calcularTaxa`: adicionar um novo bloco `else if (tipoConta.equals("pj")) { ... }` com as regras de TED, PIX, SAQUE e a regra extra dos funcionários (~10 linhas).

No método `descreverTransacao`: o `substring(0,1).toUpperCase()` pode não ser suficiente para nomes compostos como "pj" → "PJ", exigindo tratamento adicional.

Estimativa: ao menos 12 linhas novas, todas dentro de uma classe já existente, misturadas com código de outras contas — um risco alto de introduzir regressões.

4. Se dois programadores precisarem alterar regras de contas diferentes ao mesmo tempo, eles conseguem trabalhar em paralelo?

Não. Todo o código está concentrado em uma única classe `BancoLegado`. Se o Programador A alterar a regra da Conta Black e o Programador B alterar a regra da Conta Universitária

simultaneamente, ambos editarão o mesmo arquivo, gerando inevitavelmente conflito de merge no sistema de controle de versão (Git, SVN, etc.).

5. O princípio do Aberto/Fechado (Open/Closed) está sendo respeitado? Por quê?

Não está sendo respeitado. O Princípio Aberto/Fechado (OCP) determina que uma classe deve estar aberta para extensão (novas funcionalidades podem ser adicionadas) e fechada para modificação (o código existente não deve ser alterado para isso). No sistema legado, toda nova funcionalidade (seja um novo tipo de conta ou um novo tipo de transação) exige modificar diretamente a classe BancoLegado, aumentando o risco de quebrar comportamentos já testados e funcionando.

3 Violações dos Pilares da POO

Violação 1 — Encapsulamento quebrado

As regras de negócio de cada tipo de conta estão expostas e misturadas dentro de um único método da classe BancoLegado. A lógica de "como a Conta Black calcula sua taxa" deveria estar encapsulada dentro da própria Conta Black, inacessível e invisível para o restante do sistema.

Violação 2 — Ausência de Polimorfismo

O código usa uma cadeia de if-else para selecionar comportamento com base em uma String. Isso é exatamente o problema que o polimorfismo resolve: cada objeto responde à mesma mensagem (calcularTaxa) de maneira própria, sem que o chamador precise saber com qual tipo de conta está lidando.

Violação 3 — Violação da Responsabilidade Única (SRP)

A classe BancoLegado acumula múltiplas responsabilidades: conhece e implementa as regras de todos os tipos de conta, além de formatar a descrição das transações. Pelo SRP, cada classe deveria ter apenas um motivo para mudar. Aqui, a classe muda se qualquer regra de qualquer conta mudar — e também se o formato da descrição mudar.

2 Melhorias Iniciais Propostas

Melhoria 1 (imediata, sem refatoração estrutural):

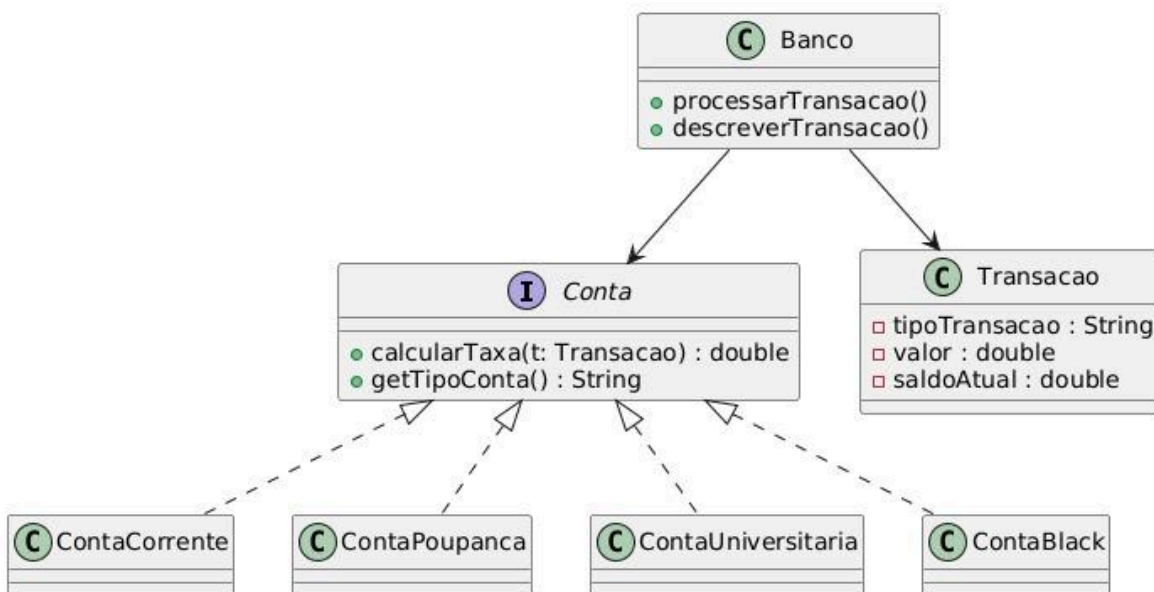
Substituir `tipoConta.equals("corrente")` por `tipoConta.equalsIgnoreCase("corrente")` em todas as comparações. Isso elimina imediatamente o bug de case-sensitivity sem alterar a estrutura do código.

Melhoria 2 (primeira etapa da refatoração):

Criar classes separadas para cada tipo de conta (ContaCorrente, ContaPoupanca, etc.), cada uma responsável por calcular sua própria taxa. Isso é o passo inicial para aplicar o polimorfismo e distribuir corretamente as responsabilidades.

FASE 2 — PROJETO

Diagrama de Classes - UML



Justificativa 1 — "Optamos por usar INTERFACE porque..."

Optamos por usar INTERFACE porque ela define apenas o contrato (aquilo que todo tipo de conta deve ser capaz de fazer (calcularTaxa e getNomeConta)) sem impor nenhuma implementação concreta. Como cada tipo de conta possui regras de negócio completamente distintas entre si, não existiria código reutilizável significativo em uma superclasse comum. A interface garante máxima flexibilidade: cada implementação é totalmente livre para aplicar sua lógica exclusiva, sem herdar comportamentos indesejados de uma classe pai. Além disso, o uso de interface torna o sistema altamente extensível — qualquer nova conta pode ser adicionada apenas criando uma nova classe que implemente o contrato, sem tocar em nada existente.

Justificativa 2 — "Optamos por NÃO usar herança de implementação (classe abstrata concreta) porque..."

Optamos por NÃO usar herança de implementação porque os quatro tipos de conta não

compartilham nenhum comportamento padrão comum que justifique um método concreto em uma superclasse. Uma classe abstrata com métodos concretos só seria adequada se houvesse lógica reutilizável entre as subclasses — o que simplesmente não ocorre aqui: a Conta Corrente usa valores fixos por tipo de transação, a Conta Black ignora o tipo de transação e olha para o saldo, a Conta Universitária aplica um limite condicional, e assim por diante. Usar herança apenas para "reaproveitar o tipo" seria herança de conveniência, um anti-padrão que criaria acoplamento desnecessário entre as classes, dificultaria futuras mudanças e tornaria a hierarquia mais frágil.

Justificativa 3 — "Se uma Conta PJ for criada no futuro, nossa arquitetura exige apenas..."

Se uma Conta PJ for criada no futuro, nossa arquitetura exige apenas a criação de uma nova classe ContaPJ que implemente a interface Conta, implementando os dois métodos do contrato (calcularTaxa e getNomeConta) com as regras específicas da conta PJ. Nenhuma classe existente precisa ser modificada — nem a interface, nem as outras contas, nem a classe Main ou Transacao. O sistema é imediatamente capaz de operar com o novo tipo de conta por meio do polimorfismo. Isso demonstra o pleno respeito ao Princípio Aberto/Fechado: aberto para extensão, fechado para modificação.